

Application Mobile avec Capteurs (AMC)

✓ Mini-projet 2026

Table des matières

1. Objectif
2. Livrables
3. Échéances et jalons obligatoires
4. Notation
5. Politique d'utilisation des outils IA

✓ 1. Objectif

L'objectif de ce mini-projet est de développer une application Python-Kivy **originale**, qui montre votre capacité à aller au-delà des connaissances acquises lors des mini-projets du cours. Le thème de l'application est **libre**, mais elle devra respecter les contraintes suivantes :

- Une interface graphique intuitive, pratique et soignée.
- L'intégration d'au moins un capteur autre que l'écran tactile, comme une caméra, un accéléromètre, un micro, ou une fonctionnalité particulière comme l'échange de données entre applications.
- L'ajout de fonctionnalités **non abordées en cours**, pour démontrer votre capacité à vous former à de nouveaux modules. Voici des exemples : communication Bluetooth, reconnaissance vocale, export PDF, etc.
- L'application devra fonctionner à la fois sur ordinateur **et** sur tablette. Vous pouvez aussi implémenter deux applications complémentaires (une pour l'ordinateur et une pour la tablette).

Les compétences développées lors de ce mini-projet vous seront utiles pour vos futurs projets de semestre, de Bachelor, de Master ou tout autre projet d'ingénierie, où vous devrez parfois développer des outils logiciels avec interface graphique pour faciliter l'utilisation de vos programmes.

Ce mini projet peut être réalisé **en binôme ou en monôme**. Pour les projets en binôme, la quantité de travail effectué devra être suffisante pour deux étudiants.

Il est important que votre application se démarque clairement des exemples disponibles en open source et des projets réalisés précédemment dans le cadre de ce cours (mini-projets du cours et d'autres étudiants).

Les objectifs de votre application devront être ambitieux tout en étant réalisables dans le temps imparti et en fonction de votre niveau de compétences en développement. Pour cette raison, il est recommandé de choisir des fonctionnalités et des objectifs qui peuvent être atteints dans le cadre du projet. Si en cours de projet, vous réalisez que vos objectifs initiaux sont trop ambitieux, impossibles à atteindre ou au contraire trop simples, vous serez autorisés à adapter votre cahier des charges.

Pour vous aider à trouver un sujet qui vous intéresse, vous pouvez consulter les travaux des années précédentes :

Vidéos des projets réalisés entre 2020 et 2025 :

2020: <https://tube.switch.ch/channels/1161cc13>

2021: <https://tube.switch.ch/channels/YLTIN9rQn4>

2022: <https://tube.switch.ch/channels/ploMoMPkaA>

2023: <https://tube.switch.ch/channels/CT9v1UbSG8>

2024: <https://tube.switch.ch/channels/LSfl4QxGbC>

2025: <https://tube.switch.ch/channels/1plpquOTPi>

Codes des projets réalisés entre 2019 et 2025 :

2019: <https://gitedu.hesge.ch/amc/amc19>

2020: <https://gitedu.hesge.ch/amc/amc20>

2021: <https://gitedu.hesge.ch/amc/amc21>

2022: <https://gitedu.hesge.ch/amc/amc22>

2023: <https://gitedu.hesge.ch/amc/amc23>

2024: <https://gitedu.hesge.ch/amc/amc24>

2025: <https://gitedu.hesge.ch/amc/amc25>

✓ 2. Livrables

Vous devez rendre les éléments suivants pour ce mini-projet :

Cahier des charges (*rendu sur Cyberlearn*)

À remplir via le formulaire sur Cyberlearn. Vous pouvez vous inspirer du document `AMC_MINIPROJET_Cahier_Des_Charges_Despont_Riedhauser.pdf` disponible sur Cyberlearn, mais assurez-vous de **ne pas faire de copier-coller**. Votre cahier des charges doit présenter le type de projet, le contexte, les objectifs, les références, la répartition des tâches (pour les binômes), etc.

Note : le respect des délais et une bibliographie pertinente sont pris en compte dans l'évaluation de ce document. Il n'est pas recommandé de citer le cours AMC ou des livres généraux sur Python et Kivy, mais cherchez des références vraiment utiles pour votre projet. Il est recommandé d'utiliser Zotero pour introduire les références dans le fichier Word selon un format respectant la norme des citations.

Exemples de tutoriels sur Zotero :

<https://bibliotheques.u-bordeaux.fr/Se-former/Les-tutoriels/Les-tutos-Zotero>

https://libguides.biblio.usherbrooke.ca/zotero/utiliser_dans_word

Dépôt gitedu (<https://gitedu.hesge.ch/amc/amc26>)

Un dépôt vous sera attribué après validation de votre cahier des charges. Tous les documents rendus sur gitedu doivent apparaître dans la branche par défaut (**master**).

Le dépôt doit contenir :

- `README.md` — Description du projet et guide utilisateur, contenant :
 - Objectifs du projet
 - Capteur ou fonctionnalité particulière choisie
 - Nouvelles fonctionnalités non vues en classe
 - Guide utilisateur (avec captures d'écran et/ou schéma fonctionnel)
 - Liste des problèmes/bugs connus
 - Travaux futurs
 - **Liste des outils IA utilisés pour réaliser ce projet et dans quel but**
 - Références avec un format respectant les normes de citation
- `JOURNAL.md` — Journal de bord du projet (voir section [Jalons](#))
- **Cahier des charges mis à jour** en format `.pdf`
- **Code source** de l'application ainsi que les fichiers annexes nécessaires

Attention : Ne rendez **pas** votre code sous la forme d'un dossier compressé.

Nom	Dernière validation
 BringMoneyAtHome.zip	Upload New File

Rendu du code sur git sous la forme d'un dossier compressé : non acceptable !

Vidéo de démonstration (*rendu sur SWITCHtube*)

Une vidéo de démonstration de votre application sur `tube.switch.ch`, présentant son objectif, ses principales fonctionnalités et le résultat final (durée recommandée : **environ 2 minutes**).

Important : Vérifiez **au moins 1 semaine avant le délai** que vous avez bien accès au canal SWITCHtube. Les vidéos rendues via un autre canal seront refusées.

Journal de bord — `JOURNAL.md`

Le fichier `JOURNAL.md` doit être **mis à jour chaque semaine de travail** sur le projet (à partir de la semaine du 12 mai) et poussé sur gitedu. Il sera consulté chaque mardi pour suivre l'avancement.

Chaque entrée doit contenir :

- La **date**
- Ce qui a été **réalisé** cette semaine
- Les **difficultés rencontrées**
- Ce qui est **prévu pour la semaine suivante**

Exemple de format :

```
## Semaine du 12 mai 2026
**Réalisé :** Mise en place de l'interface principale, intégration de la caméra.
**Difficultés :** Problème de permission Android pour l'accès à la caméra sur ta
**Prévu :** Ajouter le traitement d'image et tester sur tablette.
```

Un journal vide, non mis à jour ou mis à jour après les échéances sera pénalisé dans la note de documentation.

✓ 3. Échéances et jalons obligatoires

Le projet comporte **trois jalons obligatoires**. Ces jalons permettent de s'assurer que le projet avance régulièrement et de vous apporter un retour en temps utile.

En parallèle, le **journal de bord** et le code correspondant devra être mis à jour après chaque cours pour documenter l'avancement.

Jalon J1 — Dimanche 10 mai 2026 à 23:59 (*Cahier des charges*)

Livable : Cahier des charges à rendre sur Cyberlearn **avant le début du cours**.

Un seul cahier des charges est demandé par binôme. Sa réception conditionne l'attribution d'un dépôt GitEdu (créé par la responsable du cours) ainsi que l'ouverture d'un canal privé sur Teams (pour le feedback et la défense).

Attention : Les excuses du type «*je pensais que mon binôme avait rendu*» ou «*je voulais votre validation avant de rendre*» ne seront pas acceptées.

Du mardi 12 mai au mardi 2 juin 2026 (*Suivi via journal de bord*)

Chaque semaine sur gitedu : Le fichier `JOURNAL.md` ainsi que le code correspondant doit être mis à jour et committé chaque semaine jusqu'à la semaine de la présentation orale. La mise en forme du fichier sont laissées libres.

Suivi : Le journal de bord de chaque projet sera consulté chaque **lundi, veille du cours**, à n'importe quelle heure de la journée. Le journal devra donc être mis à jour au plus tard le **dimanche avant 23h59**. Je consulterai également le **code correspondant** afin de vérifier la cohérence entre le travail rapporté et les développements effectués. Si le journal indique un manque d'avancement, un retard important, ou si aucune mise à jour n'est présente, un **entretien en présentiel sera imposé** aux personnes concernées pour faire le point sur l'avancement du projet.

L'absence de progression visible dans le journal sera prise en compte dans l'évaluation finale (application et documentation).

Jalon J2 — Semaine 17 (15–18 juin 2026) (*Présentation et démonstration*)

Avant l'heure prévue pour votre présentation, vous devez avoir rendu votre projet complet sur <https://gitedu.hesge.ch/amc/amc26>

Présentation et démonstration des applications selon l'ordre de passage. La présentation aura lieu **à distance sur Teams dans un canal privé** et sera enregistrée pour l'évaluation.

Durée : environ 5 minutes de présentation par personne + 5 minutes de démo et questions (ajusté selon le nombre de projets).

Chaque membre du binôme devra être en mesure de répondre à **toute question** sur le projet, y compris les parties développées par l'autre membre ou par une IA.

Attention :

- Votre application devra être terminée et **entièrement fonctionnelle** pour la démonstration.
- **Aucun point ne sera attribué** pour cette partie si vous manquez votre heure de passage.
- Si la démonstration ne peut pas être réalisée en direct, prévoyez une vidéo.
- Vous pouvez utiliser [scrcpy](#) pour afficher l'écran de la tablette.

Jalon J3 — Lundi 22 juin 2026 avant 9h00 (*Vidéo*)

Rendre la vidéo du projet sur <https://tube.switch.ch/channels/KsMYGSwMdB>

Attention vous devez vous connectez au moins une fois sur <https://tube.switch.ch/organizations/20> avant le **dimanche 10 mai 23:45** afin que la responsable du cours puisse vous donner accès au canal.

Vérifiez **avant le lundi 15 juin** que vous avez bien accès au canal. Pour les binômes, les deux membres doivent avoir accès au canal.

Rappel: Sauf pour le cahier des charges, **aucun point ne sera accordé** aux parties rendues hors délai, au mauvais endroit ou de la mauvaise manière. L'exception pour le cahier des charges est due au fait qu'il est nécessaire pour démarrer le projet mais **la moitié des points sera automatiquement retirée** si le délai n'est pas respecté.

✓ 4. Notation

Note finale du cours AMC :

$$\text{Note finale} = 0.3 \times \text{Note 1 (Quiz)} + 0.7 \times \text{Note 2 (Mini-projet)}$$

Récapitulatif des points du mini-projet

Livrable	Points	Remarque
Cahier des charges (CDC)	0.5 pt	Rendu sur Cyberlearn (J1)
Journal de bord (<code>JOURNAL.md</code>)	0.5 pt	Mis à jour chaque semaine sur gitedu
Présentation et démonstration	2 pts	Teams enregistré, semaine 17
Documentation (<code>README.md</code>) + CDC mis à jour)	1 pt	Sur gitedu
Vidéo	1 pt	Sur SWITCHtube
Code source avec versionning	1 pt	Sur gitedu
Application (qualité, originalité, fonctionnement)	5 pts	Selon grille détaillée ci-dessous
Total	11 pts	

Attention: Une application **non fonctionnelle ou insuffisante** entraînera une perte de points sur les critères présentation, documentation, vidéo et code.

Cahier des charges (0.5 pt)

`AMC_MINIPROJET_2026_Cahier_Des_Charges_Noms.docx` — à rendre sur Cyberlearn.

Critères d'évaluation :

Critère	Détail
Respect du délai	Retard = perte automatique de la moitié des points
Qualité de la description	Contexte clair, objectifs précis, différent de l'énoncé
Bibliographie	Références pertinentes, formatées correctement (Zotero recommandé), citées dans le t
Répartition des tâches	Obligatoire pour les binômes
Orthographe	Plusieurs fautes = pénalité

Principales erreurs des années précédentes : Rendu en retard, mauvais format des références, références bateau (cours AMC, documentation générale Python/Kivy), description copiée de l'énoncé, pas de répartition des tâches pour les binômes, copier-coller du cahier des charges exemple.

Journal de bord `JOURNAL.md` (0.5 pt)

Critères d'évaluation :

Critère	Détail
Régularité des mises à jour	Une entrée par semaine de travail, commitée sur gitedu
Contenu des entrées	Réalisé / difficultés / prévu — pas juste une ligne
Cohérence avec le projet rendu	Le journal doit refléter le travail réel effectué

Barème indicatif : 0.5 pt si le journal est complet et régulier / 0.25 pt si partiel / 0 pt si absent ou vide.

Présentation et démonstration (2 pts)

Présentation sur Teams (enregistrée), environ 5 min de présentation + 5 min de démo et questions.

Critères d'évaluation :

Critère	Points	Détail
Qualité de la présentation	0.5 pt	Support visuel clair, présentation préparée, langage adapté, durée respo
Démonstration de l'application	0.5 pt	Application fonctionnelle, démo complète des fonctionnalités
Réponses aux questions	1 pt	Maîtrise du code, capacité à expliquer les choix techniques

Principales erreurs des années précédentes : En retard à la présentation, pas de support visuel, présentation improvisée ou trop courte, application qui crash, pas de fichier `.apk`, application trop proche d'un mini-projet du cours, mauvaise gestion des erreurs, un membre du binôme ne connaît pas la partie de l'autre, bugs non mentionnés, langage trop familier, 5 minutes sur le contexte sans montrer l'application.

Documentation (1 pt)

Critères d'évaluation :

Critère	Points	Détail
<code>README.md</code> complet	0.6 pt	Tous les points listés dans la section Livrables sont présents, avec images
Cahier des charges mis à jour	0.2 pt	En format <code>.pdf</code> sur gitedu
Orthographe et mise en page	0.2 pt	Mise en page soignée, peu ou pas de fautes

Principales erreurs des années précédentes : Pas d'image ni de schéma fonctionnel, mise en page minimale, points manquants, README rendu en PDF au lieu de `.md`, pas de cahier des charges mis à jour, liste des outils IA absente.

Vidéo (1 pt)

Vidéo de présentation de l'application sur `tube.switch.ch`.

Critères d'évaluation :

Critère	Points	Détail
Présence et accessibilité	0.3 pt	Rendue dans le bon canal, dans les délais, accessible
Contenu : démonstration de l'application	0.5 pt	Démo complète des fonctionnalités principales, commentée
Qualité de la présentation orale ou textuelle	0.2 pt	Commentée oralement ou avec sous-titres/annotations

Principales erreurs des années précédentes : Pas de vidéo car pas d'accès à Switch, vidéo envoyée par email, vidéo non commentée, vidéo qui présente uniquement le contexte sans montrer l'application.

Code source (1 pt)

Rendu sur gitedu.hesge.ch, branche **master**.

Critères d'évaluation :

Critère	Points	Détail
Qualité du code	0.4 pt	Lisible, sans code mort, noms de variables pertinents, pas de mélange français/angl
Commentaires	0.3 pt	Commentaires utiles (pas ligne par ligne), headers de fonctions/méthodes présents
Organisation sur git	0.3 pt	Fichiers bien organisés en dossiers, historique de commits visible, pas de fichiers in

Principales erreurs des années précédentes : Code non rendu sur gitedu, code dans un dossier compressé, pas visible dans la branche master, pas de commentaires ou commentaires inutiles, pas d'historique de commits, fichiers inutiles dans le dépôt, variables globales sans raison, noms mal choisis, pas de headers, mélange anglais/français.

Application (5 pts)

C'est le critère principal du mini-projet.

Critères d'évaluation :

Critère	Points	Détail
Fonctionnement général	1 pt	L'application fonctionne sans crash majeur sur PC et tablette, les fonctionnalit
Interface graphique	1 pt	Interface intuitive, soignée, adaptée à une utilisation tactile
Intégration du capteur	1 pt	Le capteur est correctement intégré et fonctionnel
Fonctionnalités nouvelles	1 pt	Au moins une fonctionnalité non vue en cours, bien intégrée
Originalité et ambition	1 pt	L'application se démarque des projets existants et du niveau des mini-projets

Principales erreurs des années précédentes : Application très proche d'un mini-projet du cours, mauvaise gestion des erreurs (crash sur valeurs invalides), widgets présents mais non fonctionnels, application non fonctionnelle sur tablette, résultat trop simpliste par rapport aux autres projets.

✓ 5. Politique d'utilisation des outils IA

Vous pouvez utiliser des outils d'IA (ChatGPT, Claude, etc.) pour vous aider à développer votre application. Cependant, les règles suivantes s'appliquent :

Ce qui est attendu :

- Vous devez être capable d'**expliquer chaque partie de votre code** lors de la soutenance. Si vous ne pouvez pas expliquer une section, elle sera considérée comme non maîtrisée et pénalisée sur le critère *Code*.
- La liste des outils IA utilisés est **obligatoire** dans le `README.md`. Indiquez précisément pour quoi vous les avez utilisés.

Ce qui sera pénalisé :

- Un projet dont le code serait manifestement entièrement généré par IA sans maîtrise démontrée lors de la soutenance entraînera une **note insuffisante**.
- Lors de la soutenance, des questions de compréhension seront posées sur votre code. Des réponses vagues ou l'incapacité à expliquer des parties essentielles seront pénalisées.

L'objectif n'est pas d'interdire les outils IA, ils font partie de la réalité actuel du développement logiciel, mais de s'assurer que vous développez de vraies compétences en les utilisant de manière consciente et maîtrisée.